

13

Take Your App Cross-Platform with Uno Platform

Uno Platform allows developers to write XAML markup and C# code in a single code base and deploy the application to multiple platforms. Uno currently supports iOS, Android, Windows, macOS, Linux, Tizen, and the web (with **WebAssembly**). Uno Platform uses the same XAML syntax as WinUI, allowing WinUI developers to easily make the jump to Uno while reusing much of their existing XAML and C# code. We will look at how the **My Media Collection** sample app can be adapted to run on some of these other platforms with Uno Platform.

In this chapter, we will cover the following topics:

- The history of Uno Platform and discuss its current capabilities
- How to configure Visual Studio to create Uno Platform projects
- Adapting existing WinUI views and ViewModels for reuse in Uno projects
- Running and debugging an Uno Platform application on Android using the **Windows Subsystem for Android (WSA)**
- Running your application in the browser natively with WebAssembly

By the end of this chapter, you will understand how to take a WinUI application built with the Windows App SDK and port it to multiple platforms outside the Windows ecosystem with Uno Platform.

Technical requirements

To follow along with the examples in this chapter, the following software is required:

- Windows 11 version 22000.0 or later with WSA installed from the Microsoft Store. To use WSA, 16 GB of RAM is recommended.
- Visual Studio 2022 or later with the **.NET Desktop Development** workload configured for Windows App SDK development.
- If you want to build and run an iOS or macOS version of the sample application, you will need a Mac running macOS 12.5 or later with Xcode 14 or later installed from the App Store.

- To target Android devices, you can install the **.NET Multi-platform App UI development** workload in the Visual Studio Installer. This will install a supported version of the **Android SDK**.

An overview of Uno Platform

Uno Platform is an open source UI framework that installs as an extension to Visual Studio. It is cross-platform, with the ability to target Windows, iOS, Android, macOS, Linux, and WebAssembly. With a single C# and WinUI XAML code base, you can target all of these platforms. While the Uno Platform team recommends Visual Studio for the best experience, you can build Uno applications with Visual Studio Code or the **JetBrains Rider** IDE. The pros and cons of each development environment are discussed on Uno Platform's **Get Started** documentation:

<https://platform.uno/docs/articles/get-started.html?tabs=windows#select-your-development-environment>.

Uno Platform was first released in May 2018 and has been growing in popularity in recent years. They have also added to their supported platforms and shifted from UWP to WinUI 3 in their XAML support. This year, in their 4.10.13 release, they even added support for embedding **.NET MAUI** controls within Uno Platform apps, with support for an extensive number of third-party controls. We won't be covering .NET MAUI embedding in this chapter, but you can explore this exciting feature in their documentation:

<https://platform.uno/docs/articles/external/uno.extensions/doc/Overview/Maui/MauiOverview.html>.

Speaking of .NET MAUI, you are probably wondering why a developer may choose Uno Platform over Microsoft's cross-platform successor to **Xamarin.Forms**. One reason why WinUI developers would choose Uno Platform is *familiarity*. Uno apps are created with WinUI XAML, so there is no learning curve. The .NET MAUI XAML is slightly different than WinUI. If targeting Linux and web browsers is important to you, .NET MAUI applications cannot currently target either of these platforms, whereas Uno Platform can. If you or the designers in your company use **Figma** to create user interfaces, you'll find the Uno Platform Figma plugin will give your team a great head-start when building your next application.

Uno Platform is open source. On their GitHub repository (<https://github.com/unoplatform/uno>), you can track open issues, submit pull requests to improve the framework, or get insights into their latest releases. If you want to try Uno Platform before getting it installed and building your first app, you can open their interactive **Uno Playground** in your browser at <https://playground.platform.uno/#wasm-start>:

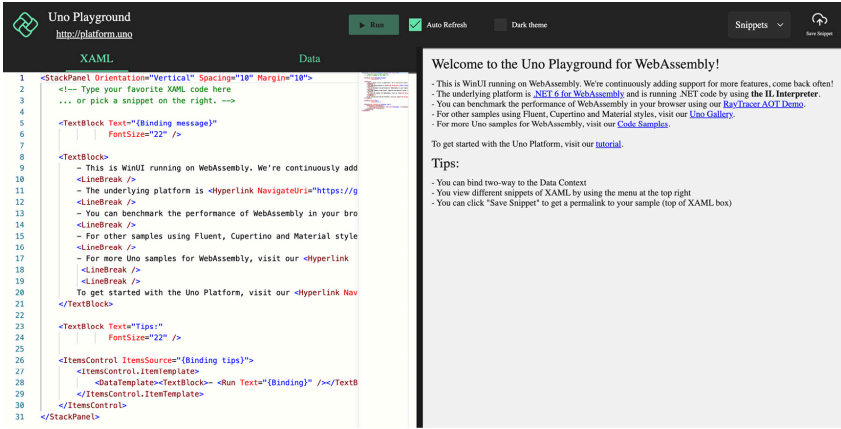


Figure 13.1 – Exploring the Uno Playground in a web browser

The Uno Playground sample app runs interactively in the browser with WebAssembly. You can make changes to the XAML in the left panel and watch it update the preview on the right in real time.

The other browser-based resource you can explore is the **Uno Gallery** (<https://gallery.platform.uno/>). In the Uno Gallery, you can explore controls, theming capabilities, and other UI and non-UI features of Uno Platform. For example, on the gallery’s page for the **Button** control, you can see how the control will be rendered in different styles – **Material**, **Fluent**, or **Cupertino** designs:

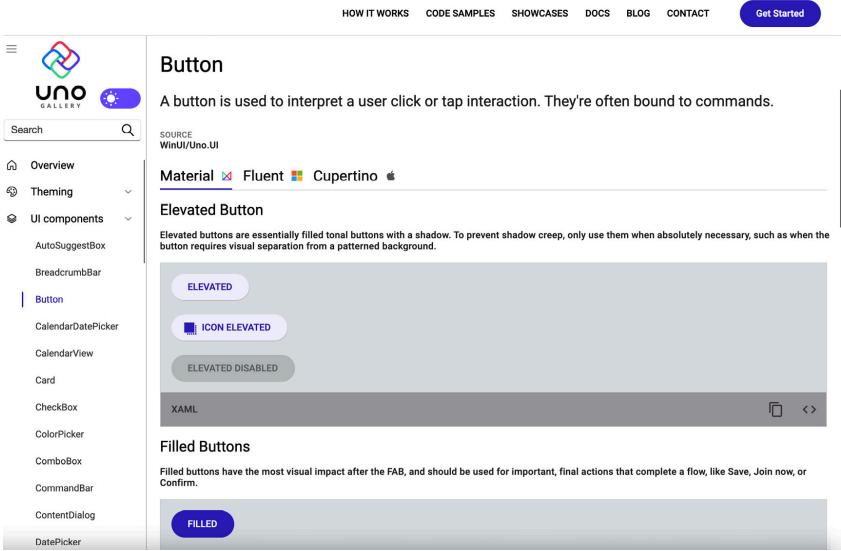


Figure 13.2 – Exploring the Button control in the Uno Gallery

Spend some time reviewing these online resources before we continue to create our first project with Uno Platform.

Creating your first Uno Platform project

In this section, we are going to create a new Uno Platform project that will be the basis of our cross-platform version of the *My Media Collection* application that we created in several of the earlier chapters of this book. Before we can create a new Uno Platform project, we need to install the extension:

1. Start by opening Visual Studio and go to **Extensions | Manage Extensions** to open the **Manage Extensions** window.
2. In the **Search** field, search for **Uno Platform**.
3. The **Uno Platform** extension should be the first result. Click **Install** and restart Visual Studio to complete the installation.
4. When you open Visual Studio again, select **Create a new project**.
5. In the **Search for templates** field, enter **Uno Platform**. You will get several results for the different Uno Platform project types.
6. Select the **Uno Platform App** template and click **Next**.
7. Name the project **UnoMediaCollection** and select **Create**. This will launch **Uno Platform Template Wizard**:

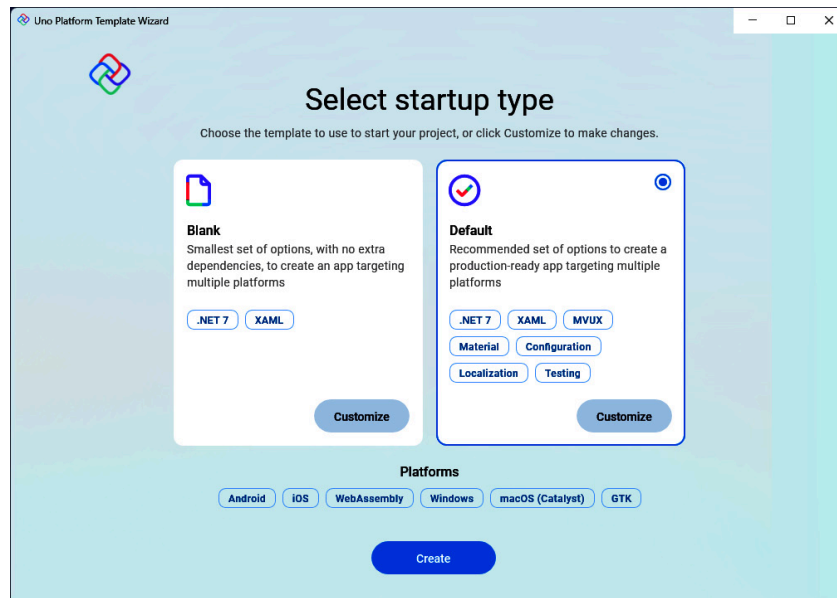


Figure 13.3 – Uno Platform Template Wizard

8. On the **Select startup type** page, select the **Customize** button on the **Default** type.
This will open the detailed steps of the wizard. From here, you can configure all of the available Uno Platform options. We will leave most of these as default, but let's step through each page.
9. On the **Framework** page, the default is currently **.NET 7.0**, but it may be **.NET 8.0** when you are reading this book. You can leave the default selection.
10. On the **Platforms** page, we are going to only work with **Windows**, **Android**, and **WebAssembly**. You can unselect the other platforms.
11. On the **Presentation** page, select **MVVM**. The default is **MVUX**, but we want to bring our ViewModel classes over from the existing

MyMediaCollection application.

12. On the **Theme** page, select **Fluent** to use the same Fluent design of the original application.
13. On the **Extensions** page, you can remove **Localization** and change **Navigation** to **Blank**.
14. You can uncheck the **Unit Tests** and **UI Tests** options on the **Testing** page. We won't get into testing in this chapter.
15. You can leave the default settings on the **Projects, Features, Authentication**, and **Application** pages. To learn more about these options, you can review the Uno Platform documentation: <https://platform.uno/docs/articles/get-started-vs-2022.html#create-an-application>.
16. Click **Create** to generate the projects and start working with them in Visual Studio. If Visual Studio prompts you to reload any projects, click **Reload**.
17. Follow the **Verify your developer environment** steps on the Uno welcome screen in Visual Studio:

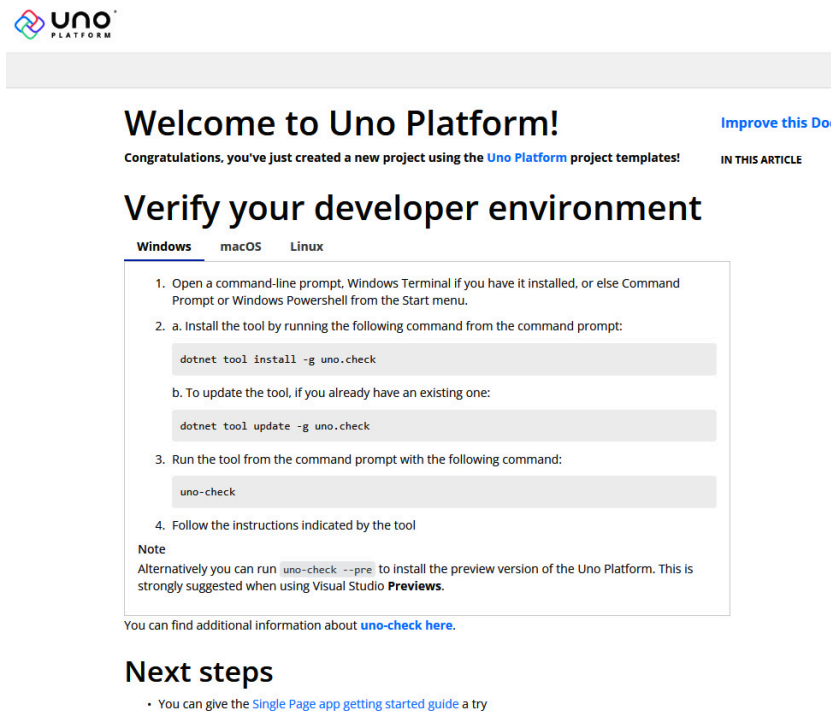


Figure 13.4 – Uno Platform welcome screen

The **uno-check** command-line utility does a great job of diagnosing potential development environment issues and automatically fixing them. You should allow it to fix any issues it identifies before continuing. You may need to reboot your system after it's done. Do this and open the project again before continuing to the next step.

18. Make sure the **UnoMediaCollection.Windows** project is set as the startup project and start debugging. The app should launch and display a window containing a **Hello Uno Platform** message:

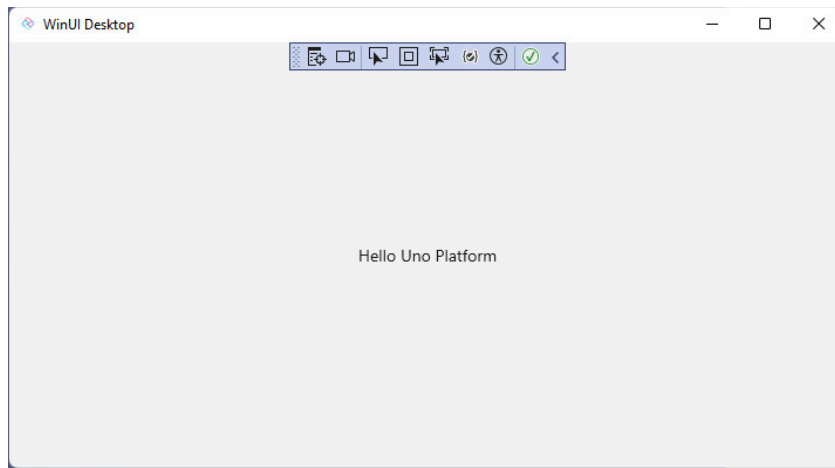


Figure 13.5 – Running UnoMediaCollection as a Windows app

That's it! We've got a running app to use as our starting point. In the next section, we will learn more about the structure of the projects as we reuse code from the **MyMediaCollection** project to enhance **UnoMediaCollection**.

Migrating WinUI XAML markup and code to Uno Platform

In this section, we are going to take the **UnoMediaCollection** solution that we created in the previous section and migrate the code from an earlier version of **MyMediaCollection**. This will give us a cross-platform version of the application that we will run on Windows, Android, and WebAssembly in the sections ahead.

In order to keep things simple for our first project, we are going to migrate the code from the completed **MyMediaCollection** solution found in the completed code for [Chapter 5, Exploring WinUI Controls](#). If you don't have a copy of that code, you can get it from GitHub here:

<https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/main/Chapter05/Complete>. The easiest way to follow along with the instructions is to download the solution, but you can also create each class in the Uno Project solution and copy and paste the WinUI project code from the GitHub editor.

We are going to use the code from [Chapter 5](#) because the SQLite database hasn't been added yet at this point. Adding code with file access to a Uno Platform project is possible but more complicated. It requires writing some **platform-specific code**. This code will be conditionally executed, depending on the platform where the application is currently running. Using SQLite on most platforms is relatively straightforward, but local file access on WebAssembly is not as easy. You can read more about platform-

specific code on Uno Platform in their documentation:

<https://platform.uno/docs/articles/platform-specific-csharp.html>.

If working offline is not a concern, the best option for data access across all these platforms is to create a lightweight web service to handle your data access. Doing this also means adding an identity solution to ensure each user is accessing their own data. Uno Platform has some documentation on consuming a web API, if you're interested in pursuing this option on your own: <https://platform.uno/docs/articles/howto-consume-webservices.html>.

Now, let's start our WinUI project migration to Uno Platform.

Migrating the WinUI project code

It's time to get started with our cross-platform transformation of **MyMediaCollection**. We'll start by importing the C# classes from the old project:

1. Start by opening the **UnoMediaCollection** solution from the previous section and create four new folders in the **UnoMediaCollection** project: **Enums**, **Interfaces**, **Model**, and **ViewModels**. We'll also be adding classes from the **Services** folder, but that folder already exists in the new project:

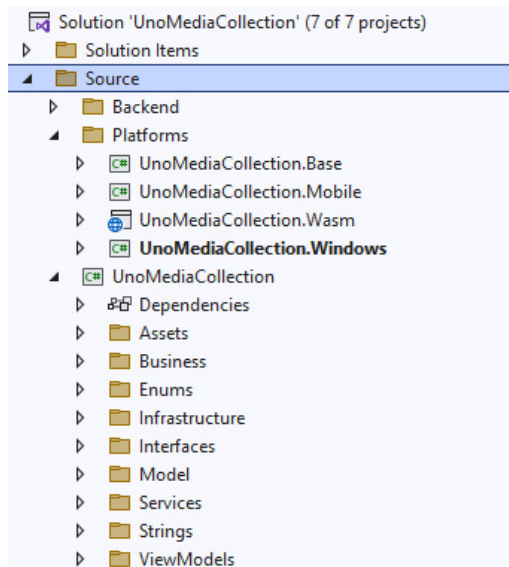


Figure 13.6 – The new folders added to the UnoMediaCollection project

2. Right-click the **Enums** folder and select **Add | Existing Item**. Browse to the **Enums** folder in the WinUI **MyMediaCollection** project, select the **ItemType.cs** and **LocationType.cs** files, and click **Add**.
3. Open each newly added file and change the **namespace** to **UnoMediaCollection.Enums**.

4. We're going to repeat these steps for each of the remaining folders.
Next, add the existing **IDataService.cs** and **INavigationService.cs** files to the **Interfaces** folder.
5. In **IDataService**, change the **namespace** to **UnoMediaCollection.Interfaces** and update the **using** statements to the following:


```
using UnoMediaCollection.Enums;
using UnoMediaCollection.Model;
```
6. In **INavigationService**, you can update the **namespace** to **UnoMediaCollection.Interfaces** and remove the **using System;** statement.
7. Add the existing **MediaItem.cs** and **Medium.cs** files to the **Model** folder.
8. Modify **MediaItem** to look like this (changes highlighted):

```
using UnoMediaCollection.Enums;
namespace UnoMediaCollection.Model
{
    public class MediaItem
    {
        public int Id { get; set; }
        public string? Name { get; set; }
        public ItemType MediaType { get; set; }
        public Medium? MediumInfo { get; set; }
        public LocationType Location { get; set; }
    }
}
```

9. Modify the **Medium** class to look like this:

```
using UnoMediaCollection.Enums;
namespace UnoMediaCollection.Model
{
    public class Medium
    {
        public int Id { get; set; }
        public string? Name { get; set; }
        public ItemType MediaType { get; set; }
    }
}
```

10. Add the existing **DataService.cs** and **NavigationService.cs** files to the **Services** folder.
11. In **DataService**, update the **namespace** to **UnoMediaCollection.Services** and update your **using** statements to only contain these three statements:

```
using UnoMediaCollection.Enums;
using UnoMediaCollection.Interfaces;
using UnoMediaCollection.Model;
```

12. In **NavigationService**, update the **namespace** to **UnoMediaCollection.Services** and change the accessor of the **AppFrame** variable from **private static** to **internal static**. We'll need to set this value from **App.cs** later. Also, update the **using** statements to contain only these two statements:

```
using UnoMediaCollection.Interfaces;
using System.Collections.Concurrent;
```

13. Add the existing **ItemDetailsViewModel.cs** and **MainViewModel.cs** files to the **ViewModels** folder.

14. In **ItemDetailsViewModel**, change the **namespace** to **UnoMediaCollection.ViewModels** and modify the **using** statements to contain these four statements:

```
using UnoMediaCollection.Enums;
using UnoMediaCollection.Interfaces;
using UnoMediaCollection.Model;
using System.Collections.ObjectModel;
```

15. In **MainViewModel**, change the **namespace** to **UnoMediaCollection.ViewModels** and update the **using** statements to contain only these four statements:

```
using Microsoft.UI.Xaml.Input;
using UnoMediaCollection.Interfaces;
using UnoMediaCollection.Model;
using System.Collections.ObjectModel;
```

All the changes so far are relatively simple. The Uno Platform project already uses the NuGet packages that we referenced in the WinUI project, so the code is very compatible.

16. Before we move on to the two views, let's make the necessary changes to **App.cs**. We need to set up **NavigationService** and register our services and ViewModel classes with the IoC container. First, add these three **using** statements to the **App** class:

```
using UnoMediaCollection.Interfaces;
using UnoMediaCollection.Services;
using UnoMediaCollection.ViewModels;
```

17. Rename the **Host** variable to **HostContainer** to match the name from our WinUI project and make it **internal static**:

```
internal static IHost? HostContainer { get; private set; }
```

18. Add the following code to the beginning of the **OnLaunched** method, right before the **builder** object is created:

```
var navigationService = new NavigationService(new Frame());
navigationService.Configure(nameof(MainPage), typeof(MainPage));
navigationService.Configure(nameof(ItemDetailsPage), typeof(ItemDetailsPage));
```

This creates the **navigationService** class, which will later be registered in the IoC container and registers the two views for navigation. I've highlighted the one difference from the code in the original WinUI project. We're temporarily passing **new Frame()** into the constructor. Later in the method, we'll set **AppFrame** to the **rootFrame** created toward the end of the **OnLaunched** method.

19. Next, update the **ConfigureServices** block in **OnLaunched** to look like this:

```
.ConfigureServices((context, services) =>
{
    services.AddSingleton<INavigationService>(navigationService);
    services.AddSingleton<IDataService, DataService>();
    services.AddTransient<MainViewModel>();
    services.AddTransient<ItemDetailsViewModel>();
})
```

This registers our classes with the IoC container just like we did in our WinUI project.

20. Update the **Host = builder.Build();** line of code in **OnLaunched** to **HostContainer = builder.Build();**.
21. Finally, immediately before the **MainWindow.Activate();** call at the end of **OnLaunched**, add this line of code to update the **AppFrame** static variable in **NavigationService**:

```
NavigationService.AppFrame = rootFrame;
```

That's everything we need to add and update except for the two views. The code won't compile successfully yet because we referenced **ItemDetailsPage** in **OnLaunched** but haven't added it yet. We'll take care of that in the next section.

Migrating the WinUI XAML views

In this section, we'll finish the additions and changes to the **UnoMediaCollection** project and run the Windows version of the application. Let's start with **ItemDetailsView**:

1. Right-click the **UnoMediaCollection** project and select **Add | New Item**.

2. In the **Add New Item** dialog, select **Uno Platform** from **C# Items** on the left panel, choose the **Page (Uno Platform Windows App SDK)** template, name it **ItemDetailsPage.xaml**, and click **Add**:

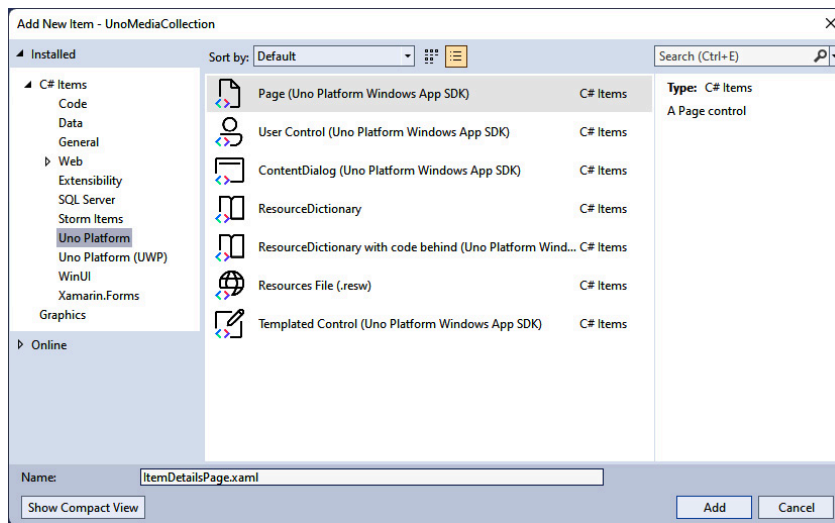


Figure 13.7 – Adding the ItemDetailsPage to the project

3. Open **ItemDetailsPage.xaml.cs** and replace the contents of the class with the following code from the WinUI project:

```
public ItemDetailsPage()
{
    ViewModel = App.HostContainer.Services.GetService<ItemDetailsViewModel>();
    this.InitializeComponent();
}
public ItemDetailsViewModel ViewModel;
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    base.OnNavigatedTo(e);
    var itemId = (int)e.Parameter;
    if (itemId > 0)
    {
        ViewModel.InitializeItemDetailData(itemId);
    }
}
```

Note that we're not migrating any of the user settings code following the **InitializeComponent** call in the constructor. We'll keep things simple for our first migration attempt.

4. You can also remove all **using** statements from the **ItemDetailsPage** class except for the following:

```
using UnoMediaCollection.ViewModels;
```

5. Open **ItemDetailsPage.xaml** and replace the child contents of the **Page** with the contents of the **Page** from the **ItemDetailsPage.xaml** in the WinUI project. Do not replace the **Page** itself because the namespaces differ.

6. Remove this **SplitButton.Resources** block because we won't be using **TeachingTip**. We removed the code behind that relies on saving user preferences to the filesystem. Without that, it would appear every time the page opens:

```
<SplitButton.Resources>
    <TeachingTip x:Name="SavingTip"
                Target="{x:Bind SaveButton}"
                Title="Save and create new"
                Subtitle="Use the dropdown button option to save your item and creat
    </TeachingTip>
</SplitButton.Resources>
```

7. Open **MainPage.xaml.cs** and add the following **using** statement:

```
using UnoMediaCollection.ViewModels;
```

8. The contents of the **MainPage** class will be similar to **ItemDetailsPage**:

```
public MainPage()
{
    ViewModel = App.HostContainer.Services.GetService<MainViewModel>();
    this.InitializeComponent();
}
public MainViewModel ViewModel;
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    base.OnNavigatedTo(e);
    if (e.NavigationMode == NavigationMode.Back)
    {
        ViewModel.PopulateData();
    }
}
```

This is slightly different than the implementation of **MainPage** in the WinUI project. We added an override of **OnNavigatedTo**. On some platforms, the list of items on **MainPage** was not updating after adding an item on **ItemDetailsPage**. Calling **PopulateData()** on **MainViewModel** when the user is navigating **Back** solved the behavior. This was an issue on both Android and WebAssembly. The Windows project worked as expected.

9. Open **MainPage.xaml** and replace the entire child contents of the **Page** from the **Page** in **MainPage.xaml**. Like we did in **ItemDetailsPage.xaml**, be careful not to replace the **Page** itself due to namespace differences.
10. Finally, add the following **using** declaration to the **Page** element in **MainPage**:

```
xmlns:model="using:UnoMediaCollection.Model"
```

The **MediaItem** model class is referenced by **DataTemplate** in our **ListView** and requires this **using** declaration.

Those are all the changes needed to get the application ready to run with Uno Platform. We didn't have to change our XAML controls at all, outside of removing **TeachingTip**.

Let's run the Windows version of the application to make sure everything works as expected. Make sure **UnoMediaCollection.Windows** is set as the startup project and run the application. It should look something like this when it launches:

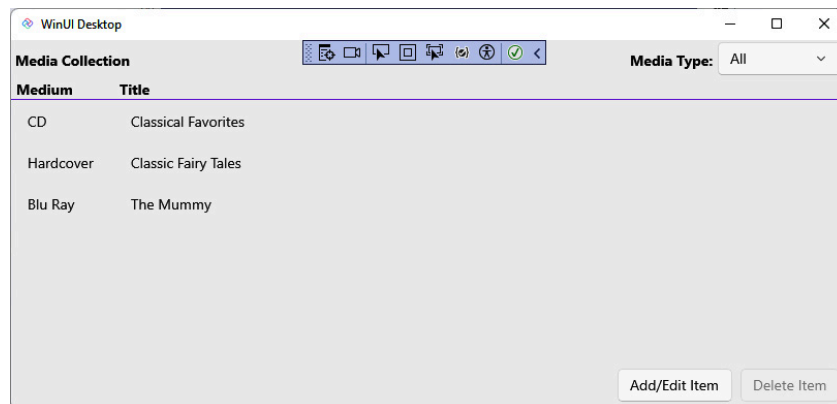


Figure 13.8 – Running the Windows project with Uno Platform

It looks exactly like the [Chapter 5](#) version of our WinUI application. If you try adding, editing, or removing items, everything should work as expected. Great work! That was pretty simple. Let's move on and try using the application on Android with WSA.

Running on Android with WSA

Running and debugging Android applications on Windows is fast and easy with WSA. The easiest way to install WSA on Windows 11 is to install **Amazon Appstore** from the Microsoft Store. You can get the app here: <https://aka.ms/AmazonAppstore>.

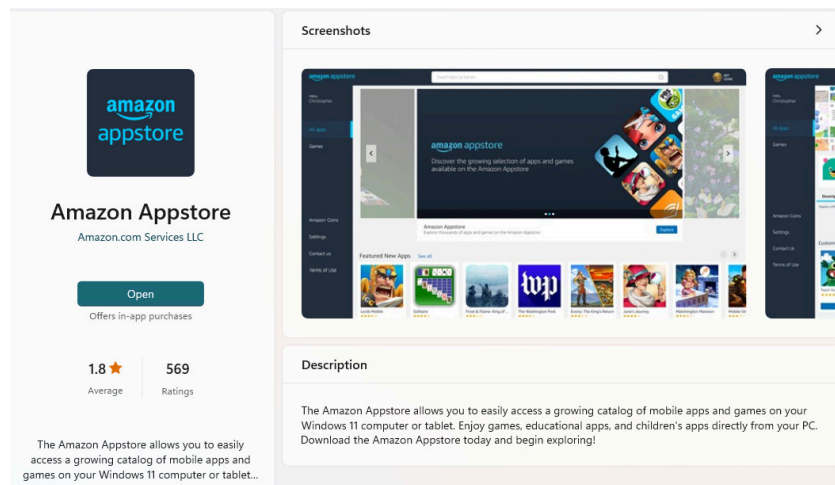


Figure 13.9 – The Amazon Appstore app in the Microsoft Store

Install the app and follow the prompts to install WSA as part of the process. When it has finished with the initial installation, you will need to restart your computer to complete the WSA installation and configuration:

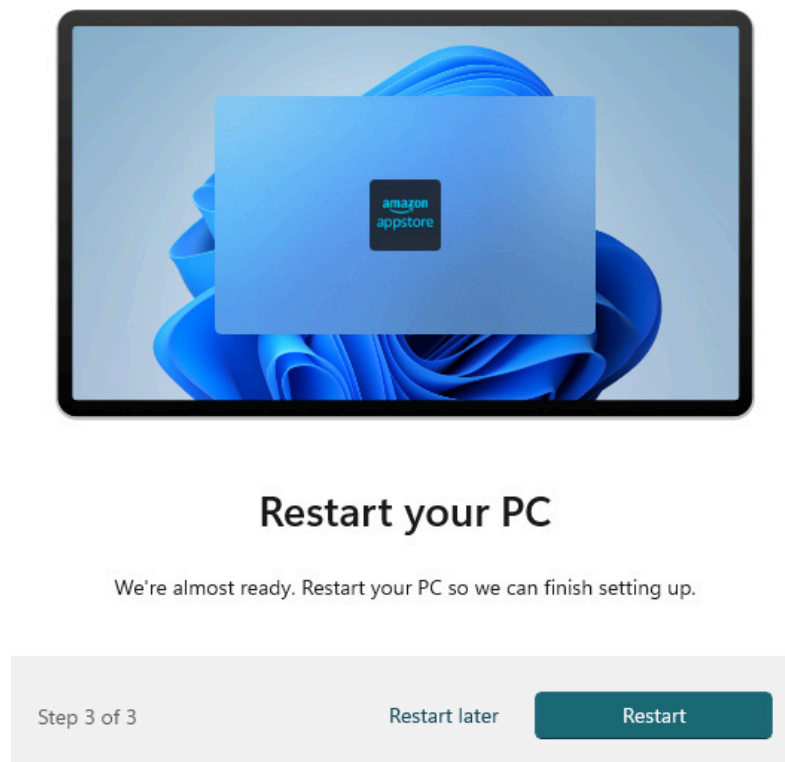


Figure 13.10 – Completing the Amazon Appstore installation

After the reboot is complete, find the Amazon Appstore app in your Windows Start menu and launch it. You will see WSA start up first:

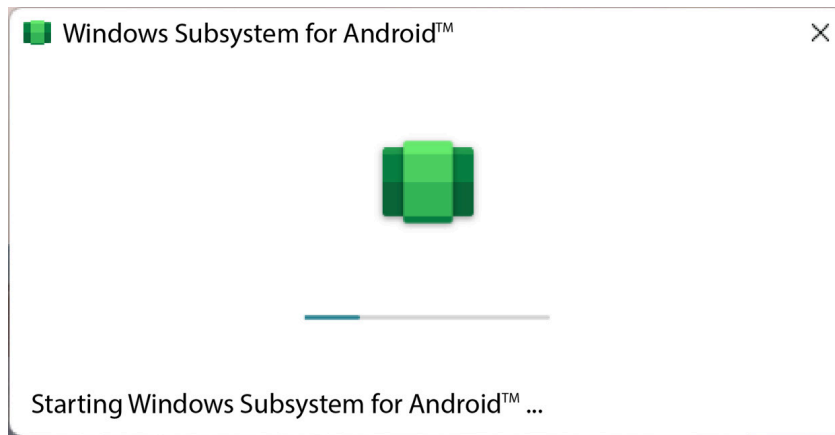


Figure 13.11 – Starting the WSA process

Keep Amazon Appstore running in the background to ensure your Android system remains active. You can minimize the window if you like. The other thing you will have to do to use WSA from Visual Studio is to ensure **Developer mode** is turned on. Launch **Windows Subsystem for Android** from the Start menu. This will open the WSA **System** settings page. Select **Advanced settings** from the left navigation panel to open the **Advanced settings** page:

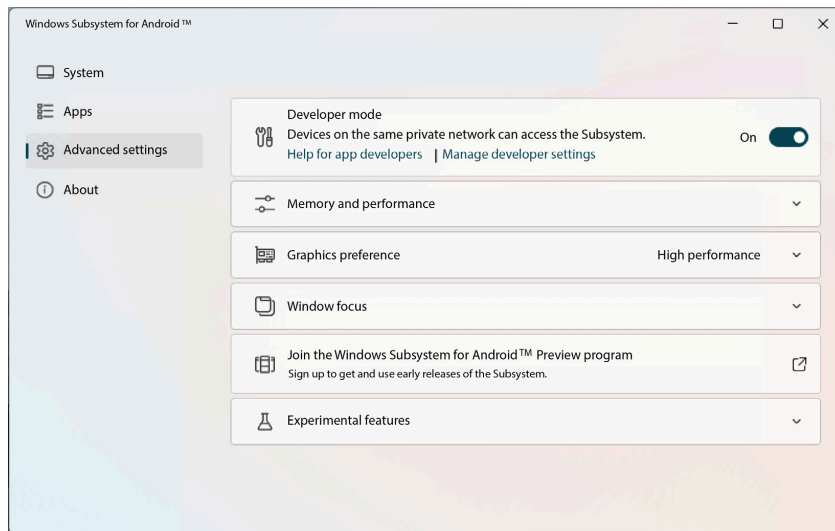


Figure 13.12 – The WSA System settings

If **Developer mode** isn't turned on, turn it on now. The last preparation step is to connect the **Android SDK** to the WSA for our debugging session. To do this, you'll need to find where your Android SDK is installed. It should be at this path if it was installed with your Visual Studio installation:

```
c:\Program Files (x86)\Android\android-sdk\
```

You will need to run the following command from a command prompt. Open a terminal or command window in the **platform-tools** subfolder

of your SDK location. This is the command if you are running within **PowerShell**. I am using **Windows Terminal** with a PowerShell window:

```
.\adb connect 127.0.0.1:58526
```

If you get the **failed to authenticate to 127.0.0.1:58526** message, check whether there is a WSA pop-up dialog behind your other windows. Click the **Allow** button on there and you should now be connected to WSA to enable debugging in Visual Studio. You will need to run this **adb connect** command each time you start debugging with WSA.

It's time to run the Android version of our application with WSA. Update the startup project to be **UnoMediaCollection.Mobile**:

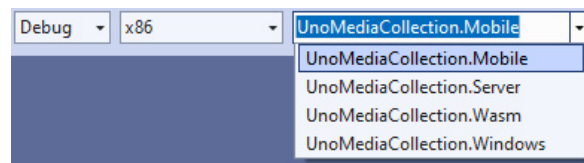


Figure 13.13 – Updating the startup project to run on mobile

If the WSA is still active, you should see **Microsoft Corporation Subsystem for Android** followed by an Android version number on the **Debug** button. Start debugging and wait a few minutes. Compiling, deploying, and running an Android application can take a bit longer than the Windows version. When the application launches, it should look something like this:



Figure 13.14 – Running the UnoMediaCollection application on Android with the WSA

Everything should function just as it did on Windows, but there could be slightly more lag in the UI, depending on the performance of your system. The cool thing about using the WSA is that you can resize your application's window to test the UI layout in different aspect ratios. Give it a try.

If you have an Android emulator configured in the Android SDK on your system, you can also try selecting that on the **Debug** button and running it there. It will look something like this on a traditional phone emulator image:

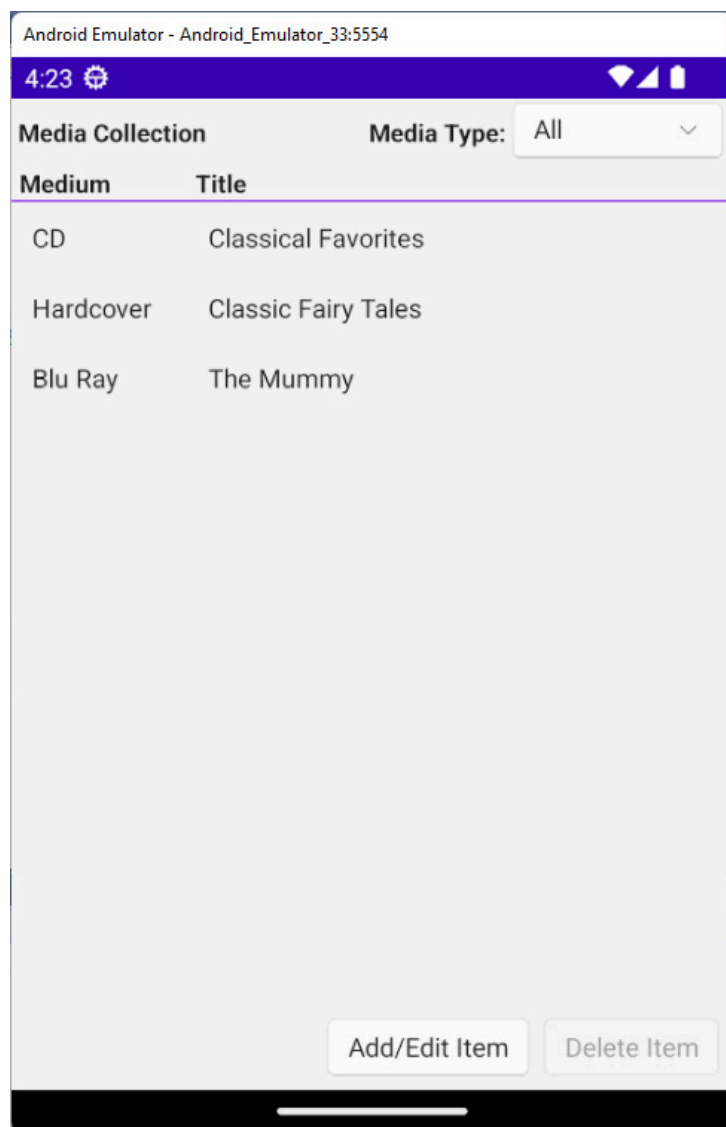


Figure 13.15 – Running the application on an Android emulator

We're up and running on Android. In many cases, switching platforms is as easy as changing the startup project. Let's finish up by trying our application on the web with WebAssembly.

Running in the browser with WebAssembly

In this final section, we will try running the application in the browser with WebAssembly. Uno Platform makes it easy to do, but like Android, the compilation and deployment can take a little time. That's because the entire application needs to run client-side within the browser. That

means that in addition to deploying our application, all its dependencies (even a version of the .NET runtime) also need to be deployed.

This is one reason why the adoption of WebAssembly hasn't been as fast as many anticipated. The first-time load performance of these apps can be sluggish at best. Uno Platform published a blog post about optimizing WebAssembly performance with Uno Platform applications. If you plan to pursue this option, you should read this post:

<https://platform.uno/blog/optimizing-uno-platform-webassembly-applications-for-peak-performance/>.

Change the startup project to **UnoMediaCollection.Wasm** and start debugging. You'll notice a command window launch in the background, hosting the web server that deploys the WebAssembly application. Next, a browser window will open. While the application is deploying and loading, you'll see a Uno Platform logo acting as a splash screen. When the application loads, it will look like this:

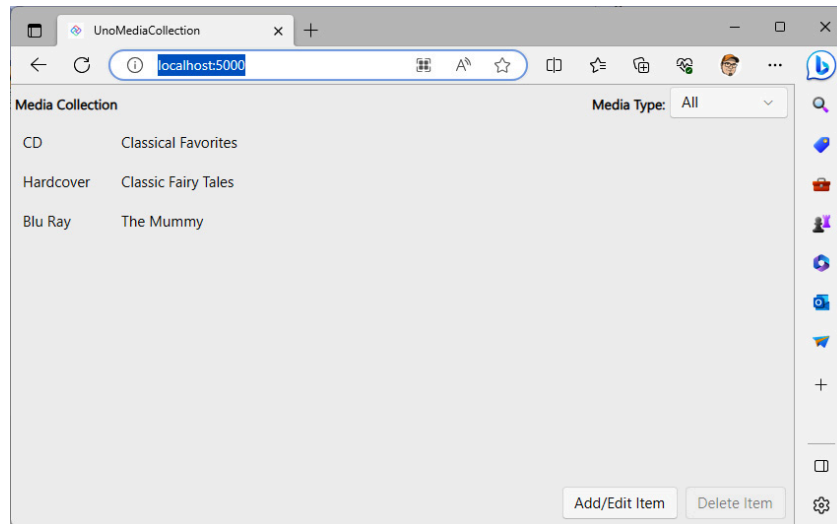


Figure 13.16 – Running the Uno Platform application in a browser with WebAssembly

Try using the application. It should function exactly as it did on other platforms. While a few of the visuals may differ from platform to platform, Uno Platform promises near-pixel-perfect applications on each supported platform.

If you would like to explore debugging further, there are some differences when debugging WebAssembly applications. The Uno Platform documentation has some great information on this:

<https://platform.uno/docs/articles/external/uno.wasm.bootstrap/doc/debugger-support.html>.

That is all we will be doing in this section with WebAssembly and Uno Platform. Let's wrap up and review what we've learned in this chapter.

Summary

In this chapter, we learned all about Uno Platform. WinUI developers can take their Windows development experience and leverage it to build applications for every platform. While we focused on Visual Studio and Windows development here, with Visual Studio, VS Code, and JetBrains Rider, you can build your Uno Platform applications from any platform. We started with a basic *Hello World*-style application and imported the code and XAML from our WinUI project to create a cross-platform version of the application with very few changes. We also learned how you can leverage WSA to debug resizable Android applications on Windows without configuring an emulator. Finally, we ran our application in the browser with Uno Platform and WebAssembly. You're ready to try Uno Platform for yourself and test some of the other platforms that it supports.

In the next (and final) chapter, we will discover how to deploy WinUI 3 applications to the Microsoft Store, via **WinGet**, or with enterprise deployment options.

Questions

1. What are two deployment options when debugging Android applications with Visual Studio?
2. What are two application design patterns supported by Uno Platform?
3. What XAML schemas are supported by Uno Platform?
4. When was the first Uno Platform release?
5. What client-side web technology can .NET and Uno Platform developers leverage to run applications natively in the browser?
6. What two online resources does Uno Platform provide to test their controls and components in your browser?
7. What design tool can you leverage to design your Uno Platform applications before developing them in your favorite IDE?